

2026.02.26.목

🕒 생성일	@2026년 2월 26일 오전 9:17
🏷️ 태그	AWS Kubernetes

목차

1. Kubernetes
2. EC2에 k3s 설치

▼ 1) Kubernetes 기초

- 컨테이너화된 애플리케이션 확장, 배포, 관리 자동화 플랫폼
- 특징: 선언적 관리. yml 파일로 선언하고 현재 상태에 맞춰 관리
 - 컨테이너가 죽으면 → 자동 재시작
 - 노드가 죽으면 → 다른 노드로 Pod 이동
 - 트래픽 증가 → HPA(Horizontal Pod Autoscaler)가 Pod 수 자동 증가
- 핵심 기능
 - 자동 복구
 - 로드밸런싱
 - 롤링 업데이트
 - 오토스케일링
 - 서비스 디스커버리
- 구조

Control Plane (마스터 노드)	→	Worker Node (여러개)
API Server		kubelet
etcd		kube-proxy
Scheduler		Container Runtime (Docker/ Containerd)
Controller Manager		Pod → Container

- Control Plane 컴포넌트

컴포넌트	역할
kube-apiserver	모든 명령의 진입점. REST API 제공. kubectl이 여기와 통신
etcd	클러스터 상태를 저장하는 분산 Key-Value DB. 진실의 원천(Source of Truth)
kube-scheduler	새 Pod를 어떤 노드에 배치할지 결정 (리소스, affinity 등 고려)
kube-controller-manager	여러 컨트롤러 실행 (ReplicaSet, Node, Job 등) → 원하는 상태 유지
cloud-controller-manager	클라우드 프로바이더(AWS/GCP 등)와 연동

- Worker Node 컴포넌트

컴포넌트	역할
kubelet	Control Plane의 지시를 받아 Pod 생성-관리. 노드의 에이전트
kube-proxy	네트워크 규칙 관리, 서비스 → Pod 트래픽 라우팅
Container Runtime	실제 컨테이너 실행 (containerd, CRI-O 등)

- 핵심 오브젝트
 - Pod
 - 최소 배포 단위
 - 1개 이상의 컨테이너 + 네트워크/스토리지

- 1개의 IP 할당 - 생성 때마다 새로운 IP
- Deployment
 - Pod를 관리하는 상위 오브젝트
 - ReplicaSet으로 Pod 개수 보장
 - 롤링 업데이트 / 롤백 담당
- ReplicaSet
 - 지정된 Pod 수 보장
 - 직접 안 쓰고 Deployment가 관리
- Service
 - Pod에 접근하는 안정적인 엔드포인트 제공
 - Service IP는 고정적
 - **ClusterIP** - 클러스터 내부 통신
 - **NodePort** - 노드 IP:포트로 외부 노출
 - **LoadBalancer** - 클라우드 LB 연동
 - **ExternalName** - 외부 DNS로 매핑
- ConfigMap/Secret
 - 설정값 분리, 저장
- Namespace
 - 클러스터 내 논리적 분리 단위
- Ingress
 - HTTP/HTTPS 라우팅 규칙. L7 로드밸런서 역할
 - Service들 앞단에서 URL 경로 기반 라우팅
- PersistentVolume(PV) / PersistentVolumeClaim(PVC)
 - Pod가 재시작해도 데이터 유지
 - PV: 실제 스토리지, PVC: Pod의 스토리지 요청
- StatefulSet
 - 상태가 있는 앱(DB 등) 배포 (Pod 이름/순서 보장)
- DaemonSet
 - 모든 노드에 Pod 1개씩 강제 배포 (로그 수집, 모니터링 에이전트 등)
- Job / CronJob
 - 일회성 작업 / 주기적 작업 실행

▼ 2) EC2에 k3s 설치, nginx pod 생성 실습

- 마스터 노드 인스턴스 1개 (ubuntu, t3.medium)
- 워커 노드 인스턴스 여러개

```
# HostNmae 수정 - 마스터 노드:
sudo hostnamectl set-hostname Server
sudo reboot

# HostNmae 수정 - 워커 노드:
sudo hostnamectl set-hostname Agent1
sudo reboot

---

# 마스터 노드에 k3s 설치
```

```

curl -sL https://get.k3s.io/ | sh -s - --write-kubeconfig-mode 644

# 설치와 동시에 k3s.service 에 권한 부여, 버전확인
systemctl status k3s.service

---

# 마스터 통신을 위한 설정
mkdir ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown -R $(id -u):$(id -g) ~/.kube
echo "export KUBECONFIG=~/.kube/config" >> ~/.bashrc
source ~/.bashrc

kubectl get nodes

---

# k3s Node Token값 / Master id 확인
# k3s 클러스터에 워커 노드를 추가하기 위해 마스터 노드에서 NODE_TOKEN값과 마스터 노드의 IP인 MASTER_IP 값을 확인

# 마스터 노드 토큰 확인
NODE_TOKEN=$(sudo cat /var/lib/rancher/k3s/server/node-token)
echo $NODE_TOKEN

# 마스터 노드 IP 확인 ([master node의 NAME] 변경)
MASTER_IP=$(kubectl get node [master node의 NAME] -o jsonpath="{.status.addresses[0].address}")
echo $MASTER_IP

---

# 워커노드 서버에서 k3s 워커 노드 설치 (서로 다른 서버)
# 반드시 마스터와 워커의 버전을 일치시켜야함
sudo curl -sL https://get.k3s.io/ | INSTALL_K3S_VERSION=v1.44.4+k3s1 K3S_URL=https://[master node IP]:6443 K3S_TOKEN=[master node TOKEN] sh -

# 마스터 노드에서 쿠버네티스 클러스터 확인
kubectl get node -o wide

---

# 마스터 노드 상태 확인
sudo systemctl status k3s.service

# journald 로그 확인
sudo journalctl -u k3s.service

# 워커 노드 상태 확인
sudo systemctl status k3s-agent.service

# journald 로그 확인
sudo journalctl -u k3s-agent.service

# pod 생성 실습
# nginx-app이라는 pod 생성하고 내부 포트 80 연결 (내부 컨테이너인 Pod포트)
kubectl run nginx-app --image nginx --port=80

# pod 목록 확장해서 표시
kubectl get pod -o wide

```

```
# nginx-app pod 앞에 service 생성 (포트포워딩, 외부 NordPort -> Service:80 -> pod:80)
kubectl expose pod nginx-app --port=80 --target-port=80 --type=NodePort

# 서비스 목록 표시
kubectl get service

# pod 삭제
kubectl delete pod nginx-app

# 서비스 삭제
kubectl delete svc nginx-app
```

? Service port가 필요한가?

다른 pod에서 서비스에 접근할 때 필요.
Pod는:

- 죽고
- 새로 생기고
- IP가 계속 바뀝니다

하지만 Service는:

- 고정된 이름
- 고정된 port
- 내부 로드밸런싱

을 제공합니다. (변경되는 pod ip 계속 추적)
—> 디커플링 구조

- yml 파일로 동작

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx-pod
spec:
  containers:
    - name: nginx-pod-container
      image: nginx:latest
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: clusterip-service-pod
spec:
  type: ClusterIP
  selector:
    app: nginx-pod
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
---
```

```

apiVersion: v1
kind: Service
metadata:
  name: nodeport-service-pod
spec:
  type: NodePort
  selector:
    app: nginx-pod
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
---
apiVersion: v1
kind: Service
metadata:
  name: loadbalancer-service-pod
spec:
  type: LoadBalancer
  selector:
    app: nginx-pod
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
---
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-replicaset
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-replicaset
  template:
    metadata:
      labels:
        app: nginx-replicaset
    spec:
      containers:
        - name: nginx-replicaset-container
          image: nginx:latest
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: cluster-service-replicaset
spec:
  type: ClusterIP
  selector:
    app: nginx-replicaset
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
---

```

```
apiVersion: v1
kind: Service
metadata:
  name: nodeport-service-replicaset
spec:
  type: NodePort
  selector:
    app: nginx-replicaset
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30081
---
apiVersion: v1
kind: Service
metadata:
  name: loadbalancer-service-replicaset
spec:
  type: LoadBalancer
  selector:
    app: nginx-replicaset
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

```
# apply
kubectl apply -f nginx-all.yaml
```